

本次针对代码库及实操体验的 6 大专项升级需求, 对代码进行了底层链路审查, 并制定了跳跃式融合重构方案。

第一部分: 针对 6 大需求的问题诊断与架构重构设计

1. 超级组卡片重构(上下联动双卡、独立参数与要领、共用休息计时)

- 现状诊断: 旧版将超级组(如 Leo Day 5 弯举与三头、Linda Day 5 侧平举与三头)写在单个卡片标题或单纯串联, 无法独立记录两个动作各自不同的重量孔位、次数, 且容易重复计时。
- 重构方案: 创建专门的复合卡片 SupersetCard.jsx。
 - 单屏上下分割(Vertical Split Deck): 上半屏渲染动作 A(如: 二头弯举), 下半屏渲染动作 B(如: 三头下压)。
 - 完全独立的数据槽位: 动作 A 与动作 B 分别拥有独立的重量步进器、次数步进器、座椅孔位记忆、以及独立的 [📏 动作要领] 闪卡入口和 [⚙️ 设置] 背卡入口。
 - 共用底部休战大键: 打卡逻辑为——做完动作 A 第 1 组 → 立刻做动作 B 第 1 组 → 点击底部共用的 GiantRestBar → 启动 120s 共享组间休息 → 休息结束自动同时推进动作 A 和动作 B 进入第 2 组!

2. “或者”动作拆分(如哈克深蹲 / 45°倒蹬机自由切换)

- 现状诊断: Day 5 中如“哈克/腿举/分腿蹲”写在同一个卡片文本中, 如果用户今天想做腿举而不是哈克, 界面仍显示哈克深蹲的重量记忆和要领, 导致历史记录与器械配重混乱。
- 重构方案:
 - 在数据层(seedPlanData.js)将带有“或者”的动作卡正式定义为主备候选卡对(Alternative Action Pairs), 赋予独立的子 ID(如 leo_d5_e1_hack 与 leo_d5_e1_legpress)。
 - 在卡片顶部加入醒目的 [↔️ 切换器械: 哈克深蹲 | 45°倒蹬机] 胶囊切换按钮。点击切换时, 卡片即刻平滑重构该器械专属的实景图解、向心离心要领、历史重量记忆与单边挂片计算, 两套记录在 LocalStorage 中互不干扰、各自沉淀。

3. 每日训练清算卡(DailySummaryCard: 中途随时查改 + 实时双向同步)

- 现状诊断: 之前只有完成所有动作滑到最后才能看总览, 且无法反向修改某组由于疲劳临时改变的重量或漏记的次数。
- 重构方案:
 - 新增 DailySummaryCard.jsx, 以扑克牌的形式存在于每日动作卡末尾(完赛尾卡之前), 同时在顶栏常驻 [📅 当日清算] 快捷抽屉入口。
 - 双向实时同步: 你在任意动作卡完成一组打卡, 清算卡表格中该行自动点亮勾选; 你在清算卡中直接点击修改第 2 组重量为 32.5kg、次数改为 9 次, 返回动作卡时即时生效并写入当日持久化日志。

4. 每日开篇「加油欢迎卡(WelcomeCard)」

- 重构方案:
 - 每日牌堆的第一张卡(Card #0)固定为专属 WelcomeCard.jsx。
 - 卡片直观展示: 当日训练主题(如 Day 3· 上肢水平推拉胸肩重装日)、当日本肌群 ACSM 核心目标、热身激活指南(如肩袖 YTWL 弹力带激活)、场馆快速确认, 以及 Leo / Linda 的个性化打气语。
 - 点击卡片正下方巨型按钮 [🔥 开始今日第 1 项训练 →] 或向上滑动, 直接翻入第一个正式动作卡。

5. ACSM 2026 重点肌群周训练量科学审查报告(Leo & Linda)

严格对照 **ACSM 2026 肥大增肌黄金标准** (每个重点刺激肌群在 **8 天微周期**内完成 $\geq 10\sim12$ 组有效组, $RIR \leq 2$) 进行全科目对账:

重点肌群	Leo 当前计划周期容量	Linda 当前计划周期容量	审查结论与精准微调措施
股四头肌 (前链)	Day 1 (7组) + Day 5 (4组) = 11组	Day 1 (3组倒蹬) + Day 5 (轻推) = 6~8组	Leo 达标; Linda 偏低 : 在 Linda Day 1 倒蹬增至 4 组, 并在商业馆 Day 1 增加保加利亚分腿蹲 3 组, 配平至 10~11组 。
臀大肌与腘绳 (后链)	Day 1 (8组) + Day 5 (3组) = 11组	Day 1 (8组) + Day 5 (6组) = 14组	双双优秀达标: Linda 臀腿重点打造, 彻底废除山羊挺身, 安全高效。
胸大肌 (水平推/夹)	Day 3 (9组) + Day 5 (3组) = 12组	Day 3 (5组) + Day 5 (3组) + 俯卧撑 = 10~11组	双双达标: 处于 10~12 组最佳刺激区间。
背部 (垂直拉/水平划)	Day 4 (11组) + Day 5 (3组) = 14组	Day 4 (8组) + Day 5 (3组) = 11组	双双达标: Leo 强化背阔与上背, Linda 采用金属宽钩死手模式, 彻底保护手腕。
三角肌 (直角肩体系)	Day 3(7组) + Day 4(3组) + Day 5(2组) = 12组	Day 3(5组) + Day 4(7组) + Day 5(5组) = 17组	超高品质达标: Linda 侧后束高频多角轰炸, 直角肩形态塑造拉满。
手臂 (二头/三头)	直系 8组 + 复合协同 = 约 13组	直系 8组 + 复合协同 = 约 11组	双双达标: Day 5 采用超级组收尾, 兼顾效率与充血。

计划修订结论: 在 seedPlanData.js 中将 Linda 商业馆 Day 1 的前链补充动作规范化, 确保商业馆与家庭馆的周期容量均严格对齐 ≥ 11 组。

6. 休息结束长响铃 + 全屏任意点击停止 / 10 秒自动推进

- 重构方案:

- 长音频引擎改造(**soundAndWakeLock.js**):使用 Web Audio API 内置递归调度器, 休息结束时播放长达 **10 秒**连续渐强金属提示音(三音阶循环 **chime**) 并伴随节奏震动。
- 全屏遮罩打断监听:响铃触发时, 全屏覆盖高透明度金绿色呼吸脉冲遮罩层:
 - 操作一:用户点击屏幕任意区域, 立刻停止发声, 并自动翻牌进入下一个动作卡或下一组;
 - 操作二:若手机放在一旁未操作, **10 秒**倒计时结束自动静音并推进到下一项, 保障训练节奏不拖沓。

第二部分:核心落地源码实现

1. 升级版长提示音与全局打断引擎:src/utls/soundAndWakeLock.js

JavaScript

```
// src/utls/soundAndWakeLock.js
```

```
class SoundAndWakeManager
```

```
  constructor()
```

```
    this.audioCtx = null
```

```
    this.wakeLock = null
```

```
    this.ringingTimer = null
```

```
    this.isRinging = false
```

```
  
```

```
  initAudio()
```

```
    if (this.audioCtx && typeof window !== 'undefined')
```

```
      const AudioContextClass = window.AudioContext || window.webkitAudioContext
```

```
      if (AudioContextClass)
```

```
        this.audioCtx = new AudioContextClass()
```

```
      
```

```
      
```

```
      if (this.audioCtx && this.audioCtx.state === 'suspended')
```

```
        this.audioCtx.resume()
```

```
      
```

```
      
```

```
  /**
```

```
   * 播放单节清脆升调金属提示音
```

```
   */
```

```
  _playSingleChimeNote(freq, startTime, duration = 0.35)
```

```
    if (this.audioCtx) return
```

```
    const osc = this.audioCtx.createOscillator()
```

```
    const gain = this.audioCtx.createGain()
```

```

    osc.type = 'triangle'
    osc.frequency.setValueAtTime(freq, startTime)

    gain.gain.setValueAtTime(0.001, startTime)
    gain.gain.exponentialRampToValueAtTime(0.3, startTime + 0.03)
    gain.gain.exponentialRampToValueAtTime(0.001, startTime + duration)

    osc.connect(gain)
    gain.connect(this.audioCtx.destination)

    osc.start(startTime)
    osc.stop(startTime + duration)
  }

  /**
   * 启动长达 10 秒的可中断循环提示音 (每 1.6 秒循环一次三连音 + 震动)
   */
  startContinuousAlarm(onAutoStop) {
    this.initAudio()
    this.stopContinuousAlarm() // 确保单例
    this.isRinging = true

    const playLoop = () => {
      if (this.isRinging || this.audioCtx) return

      // 手机马达脉冲震动
      if (typeof navigator !== 'undefined' && navigator.vibrate) {
        navigator.vibrate([250, 100, 250, 100, 350])
      }

      // 三连金属琶音 (C5 -> E5 -> G5)
      const now = this.audioCtx.currentTime
      this.playSingleChimeNote(523.25, now)
      this.playSingleChimeNote(659.25, now + 0.14)
      this.playSingleChimeNote(783.99, now + 0.28)
    }

    playLoop()
    const intervalId = setInterval(playLoop, 1600)

    // 最长 10 秒自动停止
    const timeoutId = setTimeout(() => {

```

```

    this stopContinuousAlarm()
    if (onAutoStop) onAutoStop()
  } 10000

  this ringingTimer = { intervalId, timeoutId }

  /**
   * 停止响铃 (点击屏幕任意位置或 10s 超时调用)
   */
  stopContinuousAlarm()
  this isRinging = false
  if (this ringingTimer)
    clearInterval this ringingTimer.intervalId
    clearTimeout this ringingTimer.timeoutId
  this ringingTimer = null

  playTick()
  if (typeof navigator !== 'undefined' && navigator.vibrate)
    navigator.vibrate 12

  async requestWakeLock()
  if (typeof navigator !== 'undefined' && 'wakeLock' in navigator)
    try
      this wakeLock = await navigator.wakeLock.request 'screen'
    catch (err)
      // 低电量模式忽略
  }

  export const gymDeviceManager = new SoundAndWakeManager()

```

2. 全屏轻触打断/10s 自动推进的巨型休息条:src/components/GiantRestBar.jsx

JavaScript

```
// src/components/GiantRestBar.jsx
import React, { useState, useEffect, useRef } from 'react'
import { Timer, BellRing, CheckCircle2, SkipForward, Plus } from 'lucide-react'
import { gymDeviceManager } from '../utils/soundAndWakeLock'

export default function GiantRestBar({
  activeSetNo, // 当前组号
  totalSets, // 总组数
  allSetsCompleted, // 是否所有组完成
  restDurationSeconds, // 预设秒数
  theme,
  onStartSetComplete, // 完成当前组打卡
  onAdvanceNext, // 推进至下一组或下一动作卡
  onRestStateChange // 通知父组件锁定翻牌
}) {
  const [status, setStatus] = useState('idle') // 'idle' | 'counting' | 'alarm_ringing'
  const [secondsLeft, setSecondsLeft] = useState(restDurationSeconds)
  const endTimeRef = useRef(null)
  const timerRef = useRef(null)

  // 通知锁定翻牌状态
  useEffect(() => {
    if (onRestStateChange) {
      onRestStateChange(status === 'counting' || status === 'alarm_ringing')
    }
  }, [status, onRestStateChange])

  useEffect(() => {
    clearInterval(timerRef.current)
    setStatus('idle')
    setSecondsLeft(restDurationSeconds)
  }, [restDurationSeconds])

  // 核心倒计时
  useEffect(() => {
    if (status !== 'counting') return

    const tick = () => {
      if (!endTimeRef.current) return
      const remaining = Math.max(0, Math.ceil((endTimeRef.current - Date.now()) / 1000))
      setSecondsLeft(remaining)

      if (remaining <= 0)

```

```
clearInterval(timerRef.current)
```

```
setStatus('alarm_ringing')
```

```
// 启动长达 10s 响铃;超时后自动停止并推进
```

```
gymDeviceManager.startContinuousAlarm() =>
```

```
handleStopAlarmAndAdvance()
```

```
)
```

```
timerRef.current = setInterval(tick, 250)
```

```
document.addEventListener('visibilitychange', tick)
```

```
return () =>
```

```
clearInterval(timerRef.current)
```

```
document.removeEventListener('visibilitychange', tick)
```

```
), [status]))
```

```
// 停止响铃并推进下一项
```

```
const handleStopAlarmAndAdvance = () =>
```

```
gymDeviceManager.stopContinuousAlarm()
```

```
setStatus('idle')
```

```
setSecondsLeft(restDurationSeconds)
```

```
if (onAdvanceNext) onAdvanceNext()
```

```
const handleStartRest = () =>
```

```
if (allSetsCompleted) return
```

```
gymDeviceManager.initAudio()
```

```
gymDeviceManager.requestWakeLock()
```

```
onStartSetComplete()
```

```
endTimeRef.current = Date.now() + restDurationSeconds * 1000
```

```
setSecondsLeft(restDurationSeconds)
```

```
setStatus('counting')
```

```
const handleSkip = (e) =>
```

```
if (e) e.stopPropagation()
```

```
clearInterval(timerRef.current)
```

```
handleStopAlarmAndAdvance()
```

```

const handleAdd15s = (e) =>
  if (e) e.stopPropagation()
  if (endTimeRef.current)
    endTimeRef.current += 15000
    setSecondsLeft(s => s + 15)

```

```

const progressRef = {
  status: 'counting',
  time: 0,
  min: 100,
  (restDurationSeconds - secondsLeft) /
    restDurationSeconds * 100
}

```

```

const formatTime = (sec) =>
  const min = Math.floor(sec / 60)
  const s = sec % 60
  return min < 10 ? `0${min}:${String(s).padStart(2, '0')}` : `${min}:${String(s).padStart(2, '0')}`

```

```

return (
  <div
    onClick={handleStopAlarmAndAdvance}
    className="fixed inset-0 z-[100] bg-emerald-500/35 backdrop-blur-sm flex flex-col items-center justify-center p-6 cursor-pointer animate-pulse select-none"
  >
    <div className="bg-slate-950/95 border-2 border-emerald-400 p-6 rounded-3xl text-center shadow-2xl max-w-sm w-full">
      <BellRing className="w-14 h-14 text-emerald-400 mx-auto animate-bounce mb-3" />
      <div className="text-xl font-black text-white">🔔 组间休息结束！</div>
      <p className="text-xs text-emerald-300 font-semibold mt-1">轻触屏幕任意区域立即停止响铃，进入下一项</p>
      <div className="mt-4 px-3 py-1.5 rounded-xl bg-white/10 text-[11px] text-slate-300">🕒 10秒内未操作将自动进入</div>
    </div>
  </div>
)

```

```

{ /* 休息条主体 */ }
<div

```

```

onClick={() => {
  if (status === 'idle' && !allSetsCompleted) handleStartRest();
}}
className={`relative w-full h-16 rounded-2xl overflow-hidden select-none transition-all
duration-300 border-2 shadow-xl flex items-center justify-between px-4 ${
  status === 'counting'
    ? 'bg-slate-900/95 border-cyan-400/60 text-white shadow-cyan-950/50'
    : allSetsCompleted
    ? 'bg-emerald-600/25 border-emerald-400/50 text-emerald-200'
    : `bg-gradient-to-r ${theme.accentPrimary} border-white/25 cursor-pointer
active:scale-[0.99]`}
}}
>
{status === 'counting' && (
  <div
    className="absolute inset-y-0 left-0 bg-gradient-to-r from-cyan-500/35 to-indigo-500/35
transition-all duration-300 ease-linear pointer-events-none"
    style={{ width: `${progressPct}%` }}
  />
)}
)}

```

```

{status === 'idle' && !allSetsCompleted && (
  <div className="relative z-10 w-full flex items-center justify-between">
    <div className="flex items-center gap-2.5">
      <Timer className="w-6 h-6" />
      <div className="text-left">
        <div className="text-sm font-black tracking-wide">
          完成第 #{activeSetNo} 组 · 开始 {restDurationSeconds}s 休息
        </div>
        <div className="text-[10px] opacity-85 font-medium">
          第 #{activeSetNo}/{totalSets} 组 · 结束将长响铃并自动进入下一组
        </div>
      </div>
      <span className="px-3 py-1.5 rounded-xl bg-black/25 text-xs font-black shrink-0">
        打卡休息 →
      </span>
    </div>
  </div>
)}

```

```

{status === 'idle' && allSetsCompleted && (
  <div className="relative z-10 w-full flex items-center justify-between">
    <div className="flex items-center gap-2.5">
      <CheckCircle2 className="w-6 h-6 text-emerald-400" />
      <div className="text-left">
        <div className="text-sm font-black">🎉 本动作 {totalSets} 组已全部达成！</div>
        <div className="text-[10px] opacity-80">向上滑动或查看清算卡</div>
      </div>
    </div>
  </div>
)}

```

```

    </div>
  </div>
  ))

  {status === 'counting' && (
    <div className="relative z-10 w-full flex items-center justify-between gap-2">
      <div className="flex items-center gap-2.5 min-w-0">
        <div className="text-2xl font-black font-mono text-cyan-300 tracking-tighter shrink-0">
          {formatTime(secondsLeft)}
        </div>
        <div className="text-left min-w-0">
          <div className="text-xs font-extrabold text-white truncate">
            组间休息中 · 翻牌锁定
          </div>
          <div className="text-[10px] text-amber-300 font-medium truncate">
            准备第 #{Math.min(totalSets, activeSetNo)} 组 (响铃长达10s)
          </div>
        </div>
      </div>
    </div>

    <div className="flex items-center gap-1.5 shrink-0">
      <button
        type="button"
        onClick={handleAdd15s}
        className="px-2 py-1.5 rounded-xl bg-white/10 hover:bg-white/20 text-[11px] font-bold
flex items-center gap-0.5 active:scale-95"
      >
        <Plus className="w-3 h-3" />
        <span>15s</span>
      </button>
      <button
        type="button"
        onClick={handleSkip}
        className="px-2.5 py-1.5 rounded-xl bg-amber-400 text-slate-950 text-[11px] font-black
flex items-center gap-1 shadow active:scale-95"
      >
        <SkipForward className="w-3.5 h-3.5" />
        <span>跳过进#{Math.min(totalSets, activeSetNo)}组</span>
      </button>
    </div>
  </div>
  ))
</div>
</>

```

3. 满足需求 1: 全新超级组上下联动卡片 src/components/SupersetCard.jsx

JavaScript

```
// src/components/SupersetCard.jsx
import React, { useState, useEffect } from 'react'
import { BookOpen, Settings2, Zap } from 'lucide-react'
import { SwipeNumberControl } from './SwipeNumberControl'
import { GiantRestBar } from './GiantRestBar'
import { AttachedSettingsCard } from './AttachedSettingsCard'
import { ActionCueFlashCard } from './ActionCueFlashCard'

/**
 * 满足需求1: 超级组上下联动卡片
 * 上半部: 动作 A; 下半部: 动作 B
 * 每个动作具有独立的重量、组数与要领, 共用底部休息大键
 */
export default function SupersetCard({
  exercise, // 复合超级组对象, 包含 subExercises: [exA, exB]
  venueMode,
  theme,
  cardIndex,
  totalDeckCards,
  onSaveCustomConfig,
  onResetToACSMPlan,
  onUpdateSupersetData,
  onFlipCard
}) {
  const exA = exercise.subExercises[0]
  const exB = exercise.subExercises[1]

  const variantA = exA.variants[venueMode] || exA.variants.newGym
  const variantB = exB.variants[venueMode] || exB.variants.newGym

  const [setsA, setSetsA] = useState(exA.prefillSets || [])
  const [setsB, setSetsB] = useState(exB.prefillSets || [])

  const [activeFace, setActiveFace] = useState('workout' // 'workout' | 'settings_A' | 'settings_B'
  const [activeCueEx, setActiveCueEx] = useState(null // exA | exB | null

  // 同步外部预填
  useEffect(() =>
```

```

    setSetsA(exA.prefillSets || [])
    setSetsB(exB.prefillSets || [])
  }, [exA, exB, venueMode])

  // 寻找两个动作中第一个未完成的轮次索引
  const firstIncompleteIdxA = setsA.findIndex(s => !s.completed)
  const firstIncompleteIdxB = setsB.findIndex(s => !s.completed)
  const currentRoundIdx = Math.max(0, Math.min(
    firstIncompleteIdxA === -1 ? setsA.length - 1 : firstIncompleteIdxA,
    firstIncompleteIdxB === -1 ? setsB.length - 1 : firstIncompleteIdxB
  ))
  const allCompleted = (firstIncompleteIdxA === -1 && (firstIncompleteIdxB === -1))

  const handleUpdateVal = (which, setIdx, field, val) => {
    if (which === 'A') {
      const next = setsA.map((s, i) => i === setIdx ? { ...s, [field]: val } : s)
      setSetsA(next)
      onUpdateSupersetData(exercise.id, next, setsB)
    } else {
      const next = setsB.map((s, i) => i === setIdx ? { ...s, [field]: val } : s)
      setSetsB(next)
      onUpdateSupersetData(exercise.id, setsA, next)
    }
  }

  // 共用休息大键触发:将当前轮次的动作 A 和 B 均标为完成, 并启动休息
  const handleCompleteCurrentRound = () => {
    const nextA = setsA.map((s, i) => i === currentRoundIdx ? { ...s, completed: true } : s)
    const nextB = setsB.map((s, i) => i === currentRoundIdx ? { ...s, completed: true } : s)
    setSetsA(nextA)
    setSetsB(nextB)
    onUpdateSupersetData(exercise.id, nextA, nextB)
  }

  if (activeFace === 'settings_A') {
    return (
      <AttachedSettingsCard
        exercise={exA}
        venueMode={venueMode}
        theme={theme}
        onSaveCustomConfig={onSaveCustomConfig}
        onResetToACSMPlan={onResetToACSMPlan}
        onCloseFlip={() => setActiveFace('workout')}
      />
    )
  }

```

```

    />
  
```

```

    if activeFace == 'settings_B'
      return
    <AttachedSettingsCard
      exercise={exB}
      venueMode={venueMode}
      theme={theme}
      onSaveCustomConfig={onSaveCustomConfig}
      onResetToACSMPlan={onResetToACSMPlan}
      onCloseFlip={() => setActiveFace('workout')}
    />
  
```

```

  return
  <>
    <div className={`w-full h-full rounded-3xl border p-3 flex flex-col justify-between select-none
    ${theme.cardBg}`}>
      {/* 顶部标题栏 */}
      <div className="flex items-center justify-between border-b border-white/10 pb-1.5">
        <div className="flex items-center gap-1.5">
          <span className={`px-2 py-0.5 text-[10px] font-black rounded-lg border
    ${theme.accentBadge}`}>
            {cardIndex + 1}/{totalDeckCards} · 超级组
          </span>
          <span className="text-amber-300 text-xs font-black flex items-center gap-0.5">
            <Zap className="w-3.5 h-3.5 fill-amber-300" />
            <span>双动作无缝连做</span>
          </span>
        </div>
        <span className="text-[10px] opacity-70">A+B各做一组后开始休息</span>
      </div>

      {/* 中间上下双动作联动视窗 */}
      <div className="flex-1 flex flex-col gap-2 my-1.5 overflow-hidden">
        {/* 上半部: 动作 A */}
        <div className={`flex-1 rounded-2xl border p-2 flex flex-col justify-between
    ${theme.subCardBg}`}>
          <div className="flex items-center justify-between">
            <div>
              <span className="text-[10px] font-black px-1.5 py-0.2 rounded bg-rose-500/20
    text-rose-300">
                动作 A
              </span>
            </div>
          </div>
        </div>
      </div>
    </div>
  
```

```

<h4 className="text-sm font-black mt-0.5">{variantA.name}</h4>
<div className="text-[10px] opacity-75">{variantA.machineCode}</div>
</div>
<div className="flex items-center gap-1">
  <button
    type="button"
    onClick={() => setActiveCueEx(exA)}
    className="px-2 py-1 rounded-lg bg-amber-400/20 text-amber-200 text-[10px]
font-bold flex items-center gap-0.5"
  >
    <BookOpen className="w-3 h-3" />
    <span>要领</span>
  </button>
  <button
    type="button"
    onClick={() => setActiveFace('settings_A')}
    className="p-1 rounded-lg bg-white/10 text-slate-300"
  >
    <Settings2 className="w-3 h-3" />
  </button>
</div>
</div>

{ /* 动作 A 组次列表 (紧凑展示) */ }
<div className="space-y-1 my-1 overflow-y-auto max-h-24">
  {setsA.map((s, idx) => (
    <div
      key={idx}
      className={`flex items-center justify-between gap-1 px-2 py-1 rounded-xl border text-xs
$ {
  s.completed ? theme.doneRowBg : idx === currentRoundIdx ? 'ring-1 ring-amber-400' :
'bg-black/25'
} `}
    >
      <span className="font-bold opacity-80">#{s.setNo}</span>
      <SwipeNumberControl
        value={s.weight}
        step={s.weight >= 20 ? 2.5 : 1}
        unit="kg"
        theme={theme}
        onChange={(v) => handleUpdateVal('A', idx, 'weight', v)}
      />
      <SwipeNumberControl
        value={s.reps}
        step={1}
        unit="次"
        theme={theme}
        onChange={(v) => handleUpdateVal('A', idx, 'reps', v)}

```

```

        />
        <span className={`text-[10px] font-bold px-1.5 py-0.5 rounded ${s.completed ?
'bg-emerald-500/30 text-emerald-300' : 'opacity-40'}`}>
            {s.completed ? '✓' : '待做'}
        </span>
    </div>
    )})
</div>
</div>

```

```

    {/* 下半部: 动作 B */}
    <div className={`flex-1 rounded-2xl border p-2 flex flex-col justify-between
${theme.subCardBg}`}>
        <div className="flex items-center justify-between">
            <div>
                <span className="text-[10px] font-black px-1.5 py-0.2 rounded bg-sky-500/20
text-sky-300">
                    动作 B
                </span>
                <h4 className="text-sm font-black mt-0.5">{variantB.name}</h4>
                <div className="text-[10px] opacity-75">{variantB.machineCode}</div>
            </div>
            <div className="flex items-center gap-1">
                <button
                    type="button"
                    onClick={() => setActiveCueEx(exB)}
                    className="px-2 py-1 rounded-lg bg-amber-400/20 text-amber-200 text-[10px]
font-bold flex items-center gap-0.5"
                >
                    <BookOpen className="w-3 h-3" />
                    <span>要领</span>
                </button>
                <button
                    type="button"
                    onClick={() => setActiveFace('settings_B')}
                    className="p-1 rounded-lg bg-white/10 text-slate-300"
                >
                    <Settings2 className="w-3 h-3" />
                </button>
            </div>
        </div>
    </div>

```

```

    {/* 动作 B 组次列表 */}
    <div className="space-y-1 my-1 overflow-y-auto max-h-24">
        {setsB.map((s, idx) => (
            <div
                key={idx}
                className={`flex items-center justify-between gap-1 px-2 py-1 rounded-xl border text-xs

```

```

    ${
        s.completed ? theme.doneRowBg : idx === currentRoundIdx ? 'ring-1 ring-amber-400' :
        'bg-black/25'
    }
    >
    <span className="font-bold opacity-80">#{s.setNo}</span>
    <SwipeNumberControl
        value={s.weight}
        step={s.weight >= 20 ? 2.5 : 1}
        unit="kg"
        theme={theme}
        onChange={(v) => handleUpdateVal('B', idx, 'weight', v)}
    />
    <SwipeNumberControl
        value={s.reps}
        step={1}
        unit="次"
        theme={theme}
        onChange={(v) => handleUpdateVal('B', idx, 'reps', v)}
    />
    <span className={`text-[10px] font-bold px-1.5 py-0.5 rounded ${s.completed ?
    'bg-emerald-500/30 text-emerald-300' : 'opacity-40'}`}>
        {s.completed ? '✓' : '待做'}
    </span>
    </div>
    )}]
    </div>
    </div>
    </div>

    {/* 底部共用的 GiantRestBar */}
    <div className="pt-1">
        <GiantRestBar
            activeSetNo={currentRoundIdx + 1}
            totalSets={Math.max(setsA.length, setsB.length)}
            allSetsCompleted={allCompleted}
            restDurationSeconds={exA.prescription?.restSeconds || 120}
            theme={theme}
            onStartSetComplete={handleCompleteCurrentRound}
            onAdvanceNext={() => {
                if (allCompleted && onFlipCard) onFlipCard('next');
            }}
        />
    </div>
    </div>

    {/* 要领闪卡 */}
    {activeCueEx && (

```

```

<ActionCueFlashCard
  exercise={activeCueEx}
  activeVariant={activeCueEx.variants[venueMode] || activeCueEx.variants.newGym}
  theme={theme}
  onClose={() => setActiveCueEx(null)}
/>
))
</>

```

4. 满足需求 3: 每日训练清算卡 src/components/DailySummaryCard.jsx

```

JavaScript
// src/components/DailySummaryCard.jsx
import React from 'react'
import { Calculator, CheckCircle, Save, Sparkles, ArrowRight } from 'lucide-react'
import SwipeNumberControl from './SwipeNumberControl'

/**
 * 满足需求3: 每日训练清算卡
 * 实时汇总结算当日所有动作的重量、组数与次数
 * 用户可在训练中随时打开手调, 或完赛后统一核对清算
 */
export default function DailySummaryCard({
  currentDay,
  currentUser,
  venueMode,
  dayExercises, // 当日所有动作对象列表
  todayLog, // 当日已经打卡的日志数据
  theme,
  onUpdateSingleSet, // (exerciseld, setIndex, field, value) => void
  onToggleSetDone, // (exerciseld, setIndex) => void
  onFinishWorkout, // 确认清算完毕 -> 进入完赛卡
  onReturnToDeck // 返回动作卡片流
}) {
  // 计算当日总吨位与完成率
  let totalVolumeKg = 0
  let totalPlannedSets = 0
  let totalCompletedSets = 0

  dayExercises.forEach(ex =>

```

```

const exLog = todayLog ? exercises ? find e => e.exerciseId === ex.id :
const sets = exLog ? sets || ex.prefillSets || [] :
totalPlannedSets += sets.length
sets.forEach (s) =>
  if (s.completed)
    totalCompletedSets += 1
    totalVolumeKg += Number(s.weight) * 0 + Number(s.reps) * 0
  }
})
})

return
<div className={`w-full h-full rounded-3xl border-2 p-3.5 flex flex-col justify-between select-none ${theme.cardBg}`}>
  {/* 顶部统计胶囊 */}
  <div>
    <div className="flex items-center justify-between border-b border-white/10 pb-2">
      <div className="flex items-center gap-1.5">
        <Calculator className={`w-4 h-4 ${theme.accentText}`} />
        <span className="text-xs font-black uppercase tracking-wider">
          📅 Day {currentDay} 任务结算与清算卡
        </span>
      </div>
      <span className="text-[10px] text-amber-300 font-bold">可直接点击修改任意项</span>
    </div>

    {/* 仪表盘三联卡 */}
    <div className="grid grid-cols-3 gap-1.5 my-2">
      <div className={`p-2 rounded-xl border text-center ${theme.subCardBg}`}>
        <div className="text-[10px] opacity-70">已完成组数</div>
        <div className="text-base font-black text-emerald-400">
          {totalCompletedSets}/{totalPlannedSets}
        </div>
      </div>
      <div className={`p-2 rounded-xl border text-center ${theme.subCardBg}`}>
        <div className="text-[10px] opacity-70">总负荷吨位</div>
        <div className={`text-base font-black ${theme.accentText}`}>
          {Math.round(totalVolumeKg)} <span className="text-[10px]">kg</span>
        </div>
      </div>
      <div className={`p-2 rounded-xl border text-center ${theme.subCardBg}`}>
        <div className="text-[10px] opacity-70">达成率</div>
        <div className="text-base font-black text-white">
          {totalPlannedSets > 0 ? Math.round((totalCompletedSets / totalPlannedSets) * 100) : 0}%
        </div>
      </div>
    </div>

```

```
</div>
</div>
```

```
/* 中部: 全部动作分组清单滚动视窗 */
<div className="flex-1 my-1 overflow-y-auto space-y-2 pr-1">
  {dayExercises.map((ex) => {
    const variant = ex.variants[venueMode] || ex.variants.newGym;
    const exLog = todayLog?.exercises?.find(e => e.exercisId === ex.id);
    const sets = exLog?.sets || ex.prefillSets || [];
```

```
    return (
      <div key={ex.id} className={`p-2.5 rounded-2xl border ${theme.subCardBg}`}>
        <div className="flex items-baseline justify-between mb-1.5">
          <div className="font-black text-xs text-white truncate max-w-[13rem]">
            {variant.name}
          </div>
          <span className="text-[10px] opacity-70">{variant.machineCode}</span>
        </div>
```

```
      /* 逐组清算列表 */
      <div className="space-y-1">
        {sets.map((s, sIdx) => (
          <div
            key={sIdx}
            className={`flex items-center justify-between gap-1.5 px-2 py-1 rounded-xl border
text-xs ${
              s.completed ? theme.doneRowBg : 'bg-black/35'
            }`}
          >
            <span className="font-mono font-bold opacity-80 w-6">#{s.setNo}</span>
```

```
          /* 重量调整 */
          <SwipeNumberControl
            value={s.weight}
            step={s.weight >= 20 ? 2.5 : 1}
            min={0}
            unit="kg"
            theme={theme}
            onChange={(val) => onUpdateSingleSet(ex.id, sIdx, 'weight', val)}
          />
```

```
          /* 次数调整 */
          <SwipeNumberControl
            value={s.reps}
            step={1}
            min={1}
            unit="次"
            theme={theme}
```

```

        onChange={(val) => onUpdateSingleSet(ex.id, sldx, 'reps', val)}
      />

      {/* 勾选完成状态 */}
      <button
        type="button"
        onClick={() => onToggleSetDone(ex.id, sldx)}
        className={`w-7 h-7 rounded-lg font-black text-xs flex items-center justify-center
        transition ${
          s.completed ? theme.doneBtn : 'bg-white/10 text-white/40'
        }`}
      >
        ✓
      </button>
    </div>
  )]]
</div>
</div>
);
}}}
</div>

{/* 底部操作区 */}
<div className="pt-2 border-t border-white/10 flex gap-2">
  <button
    type="button"
    onClick={onReturnToDeck}
    className="w-1/3 py-2.5 rounded-xl bg-white/10 text-xs font-bold"
  >
    返回动作卡
  </button>
  <button
    type="button"
    onClick={onFinishWorkout}
    className={`flex-1 py-2.5 rounded-xl bg-gradient-to-r font-black text-xs flex items-center
    justify-center gap-1 shadow-lg ${theme.accentPrimary}`}
  >
    <span>核对无误 · 结算入库</span>
    <ArrowRight className="w-3.5 h-3.5" />
  </button>
</div>
</div>

```

5. 满足需求 4: 每日开篇「欢迎加油卡」src/components/WelcomeCard.jsx

JavaScript

```
// src/components/WelcomeCard.jsx
```

```
import React from 'react'
```

```
import { Flame, ShieldCheck, Dumbbell, ArrowDown } from 'lucide-react'
```

```
/**
```

```
 * 满足需求4: 每日开始时的欢迎加油卡
```

```
 * 展示当日训练主题、ACSM目标及热身要领, 确认后滑入第1张动作卡
```

```
 */
```

```
export default function WelcomeCard({
```

```
  currentDay,
```

```
  currentUser,
```

```
  venueMode,
```

```
  dayMeta,
```

```
  theme,
```

```
  customAvatars,
```

```
  onStartFirstExercise
```

```
})
```

```
  const meta = dayMeta[currentDay] || {}
```

```
  const avatarUrl =
```

```
    customAvatars[currentUser] ||
```

```
    {currentUser === 'leo' ? '/images/leo-avatar.png' : '/images/linda-avatar.jpg'}
```

```
  return
```

```
    <div className={`w-full h-full rounded-3xl border-2 p-5 flex flex-col justify-between select-none ${theme.cardBg}`}>
```

```
      {/* 头部: 用户专属问候 */}
```

```
      <div className="flex items-center gap-3.5 border-b border-white/10 pb-4">
```

```
        <img
```

```
          src={avatarUrl}
```

```
          alt={currentUser}
```

```
          className="w-14 h-14 rounded-2xl object-cover border-2 border-amber-300 shadow-md"
```

```
        />
```

```
      <div>
```

```
        <span className={`text-[10px] font-black px-2 py-0.5 rounded-md border ${theme.accentBadge}`}>
```

```
          ACSM 2026 · 无极微循环
```

```
        </span>
```

```
        <h2 className="text-xl font-black mt-1">
```

```
          {currentUser === 'leo' ? '🦁 Leo, 蓄势待发!' : '🦢 Linda, 优雅开练!'}
```

```
        </h2>
```

```
        <div className="text-[11px] opacity-75 mt-0.5">
```

```
    当前场馆:{venueMode === 'newGym' ? '🌟乐刻新馆': venueMode === 'oldGym' ? '🏠传统旧馆': '🏠家庭重装'}
  </div>
</div>
</div>
```

```
{/* 中部:今日训练科目标签 */}
<div className="my-auto space-y-3">
  <div className={`p-4 rounded-2xl border ${theme.subCardBg}`}>
    <div className="flex items-center gap-1.5 text-amber-300 text-xs font-black">
      <Flame className="w-4 h-4 fill-amber-300" />
      <span>DAY {currentDay} · 核心战术主题</span>
    </div>
    <h3 className="text-lg font-black text-white mt-1">{meta.name || '核心训练日'}</h3>
    <p className="text-xs opacity-85 mt-1 leading-relaxed">
      {meta.focus || '按高神经放电速度与离心控制完成每组动作, 向心变慢即停!'}
    </p>
  </div>
```

```
{/* ACSM 2026 增肌纪律提醒 */}
<div className="p-3.5 rounded-2xl bg-indigo-500/15 border border-indigo-400/30 text-xs space-y-1.5">
  <div className="font-black text-indigo-300 flex items-center gap-1">
    <ShieldCheck className="w-4 h-4" />
    <span>ACSM 2026 执行纪律</span>
  </div>
  <div className="text-[11px] opacity-90 leading-relaxed">
    • 重点肌群冲击每周期 <b className="text-amber-200">11~12组有效组</b>[cite: 1];<br />
    • 离心阶段刹车2秒, 底部拉伸位停顿1秒卸掉反弹力[cite: 1];<br />
    • 动作间严格执行组间休息长响铃, 满血开推。
  </div>
</div>
</div>
</div>
```

```
{/* 底部启动大键 */}
<div>
  <button
    type="button"
    onClick={onStartFirstExercise}
    className={`w-full py-3.5 rounded-2xl bg-gradient-to-r font-black text-sm flex items-center justify-center gap-2 shadow-xl active:scale-[0.98] ${theme.accentPrimary}`}
  >
    <Dumbbell className="w-4 h-4" />
    <span>进入第 1 项训练卡片</span>
    <ArrowDown className="w-4 h-4" />
  </button>
  <div className="text-center text-[10px] opacity-60 mt-2">
    轻点按钮或向上滑动即可翻入动作卡片
  </div>
</div>
```

```

    </div>
  </div>
</div>

```

6. 满足需求 2 的动作卡支持器械候选切换:src/components/ExerciseCard.jsx 片段
在 ExerciseCard.jsx 的顶部渲染动作候选切换 Tab:

```

JavaScript
/* 满足需求2: 当动作包含或选器械(alternatives)时, 提供自由切换按钮 */
0
<div className="flex items-center gap-1 my-1 p-1 rounded-xl bg-black/40 border
border-amber-400/30">
  <span className="text-[10px] text-amber-300 font-bold px-1.5">⇌ 备选器械:</span>
  {exercise.alternatives.map((alt) => {
    <button
      key={alt.id}
      type="button"
      onClick={() => onSelectAlternativeExercise(exercise.id, alt.id)}
      className={`px-2 py-0.5 rounded-lg text-[10px] font-bold transition ${
        exercise.activeAltId === alt.id
          ? 'bg-amber-400 text-slate-950 font-black'
          : 'bg-white/10 text-white/70'
        }`}
    >
      {alt.name}
    </button>
  })}
</div>

```

7. 汇总总控: 重构后的主牌桌容器 src/App.jsx

将 欢迎卡 → 动作卡 (含单动作/超级组/或者切换) → 每日清算卡 → 完赛尾卡 串联为丝滑扑克牌流:

```

JavaScript
// src/App.jsx 核心牌堆组织逻辑

```

```
const deckCards = useMemo(() => {
  if (dayExercises.length === 0) return [] // 有氧日走 CardioPanel
  return [
    { type: 'welcome' }, // 卡片 0: 加油欢迎卡
    ...dayExercises.map(ex => ({ type: 'exercise' data: ex })), // 卡片 1~N: 动作卡 (普通卡或超级组卡)
    { type: 'summary' }, // 卡片 N+1: 每日清算卡
    { type: 'finale' }, // 卡片 N+2: Linda完赛卡
  ]
}, [dayExercises])
```

第三部分: 直接交给本地 **Assistant** 的执行指令
 请直接将以下指令完整发送给你的本地编程助手:

Markdown

在现有工程(gym-cb9bf3862d2c9c0ebfa25e7fa53a207b5aaaf0694)基础上执行深度重构升级, 严格实现以下6项核心功能:

1. 超级组双动作联动卡片 ('src/components/SupersetCard.jsx')

- 在数据层('seedPlanData.js')将 Leo Day 5 和 Linda Day 5 的超级组标为 `type: "superset"` 并赋予 `subExercises: [exA, exB]`
- 研发 `SupersetCard.jsx` 单屏上下拆分为两张联动子卡(动作 A 在上半部, 动作 B 在下半部)
 - 动作 A 和 B 拥有各自独立的重量调节、每组次数调节、座椅孔位记忆、以及各自独立的[动作要领]闪卡和[调]臂卡
 - 动作 A 和动作 B 共用卡片底部的 `GiantRestBar` 打卡逻辑: A 和 B 各完成一组后点击大键打卡并开始休息, 休息结束同时自动进入 A 和 B 的下一组。

2. 动作卡片“或者”选项彻底拆分为独立候选切换 ('seedPlanData.js' & 'ExerciseCard.jsx')

- 凡是“或者/或”的动作(如 Leo Day 5 哈克深蹲 或 45°倒蹬机), 在数据中拆为包含 `alternatives` 的并列项
- 在卡片顶部常驻显示 `[⇌ 备选器械: 哈克深蹲 | 45°倒蹬机]` 切换标签。点击切换时, 独立读取和保存各自的重量、组数、要领和机位记忆, 互不串台

3. 每日训练清算卡 ('src/components/DailySummaryCard.jsx')

- 新建清算卡, 排在当日最后一个动作卡之后, 完赛尾卡之前, 并在侧栏常驻 `[📊 当日清算]` 入口
- 实时统计已完成组数、总吨位(kg)、完成百分比, 以表格形式展示当日全部动作及每组重量和次数
- 用户中途或练完可随时打开, 直接手调任意一组的重量、次数或勾选完成状态, 改动实时对

向同步回动作卡并存入 LocalStorage

4. 每日开篇展示加油欢迎卡 (`src/components/WelcomeCard.jsx`)

- 每日训练的第 0 张卡片展示加油欢迎卡，展示 Leo/Linda 头像、今日训练战术主题、ACSM 周容量纪律和热身提示
- 点击卡片下方的 `[🔥 开始今日第 1 项训练 →]` 或向上滑动，直接滑入第一个动作

5. ACSM 2026 周容量严格配平审查 (`seedPlanData.js`)

- 早晨开配平 Leo 和 Linda 各重点肌群周有效组数均 $\geq 11-12$ 组。
- 重点补齐 Linda 商业馆 Day 1 的下肢前链组数，增加商业馆保加利亚分腿蹲 3 组，与家庭馆统一对齐到 11 组。

6. 休息结束长响铃引擎与全屏点击打断机制 (`soundAndWakeLock.js` & `GiantRestBar.jsx`)

- 升级 Web Audio 合成音，休息结束时播放长达 10 秒的循环升调金属提示音并伴随马达轰鸣声
- 响铃期间全屏弹出半透明金绿呼吸遮罩，用户点击屏幕任意位置立即停止发声并自动推进到下一组/下一卡，若 10 秒未操作则自动停止响铃并推进。